

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 09/Unit 09: Rule of 3

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Introduction
2. Destructor
3. Copy Constructor
4. Copy Assignment Operator
5. Summary



Introduction

- In C++, the Rule of 3 is a convention (best practice). It states that:
- If a class needs to define one of the following:
 - Destructor,
 - Copy Constructor
 - Copy Assignment Operator

then it needs to explicitly define all 3 in order to have exception safe coding.

The rule of 3 is a convention or a best practice in C++.



Rule of 3 - Destructors

As discussed in previous sessions, a **destructor** is a function that is

- called when an object goes out of scope or,
- is explicitly deleted.

It is responsible for cleaning up resources that the object may have acquired during its lifetime, such as dynamically allocated memory.



Rule of 3 – Copy Constructors

- A constructor that creates a new object as a copy of an existing object.
- It is called when an object is passed by value, returned from a function, or explicitly copied.
- If the class manages resources (like dynamic memory), a **default copy constructor** may lead to **shallow copies**, causing resource management issues.
- What is a shallow copy? It is creation of an alias of a dynamically allocated object.



Rule of 3 – Copy Assignment Operator

Copy Assignment Operator:

- This is the overloaded = operator.
- This operator is used to copy the contents of one existing object to another existing object.
- Like the copy constructor, if the class manages resources, the **default assignment operator** may lead to shallow copies.



Copy Constructor vs Copy Assignment Operator

- **Copy Constructor:** A special constructor that initializes a new object as a copy of an existing object.

```
ClassName(const ClassName& other);
```

- Called when **a new object** is created from an existing object.
- **Copy Assignment Operator:** A special assignment operator that assigns the contents of one existing object to another existing object (both objects are already created).

```
ClassName& operator=(const ClassName& other);
```

- Called when **an existing object** is assigned to another existing object.



Example – Rule of 3 (Example - followed)

```
#include <iostream>
#include <cstring>

class MyString {
private:
    char* data;

public:
    // Constructor
    MyString(const char* str = "") {
        data = new char[strlen(str)+1];

        strcpy(data, str);
    }
};
```

```
// Destructor
~MyString() {
    delete[] data;
}

// Copy Constructor
MyString(const MyString& other) {
    data = new char[strlen(other.data)+1];

    strcpy(data, other.data);
}
```




Example – Rule of 3 (Example - followed)

```
// Copy Assignment Operator
MyString& operator=(const MyString&
other) {
    if (this == &other)
        return *this;
    delete[] data;

    data = new char[strlen(other.data)
+1];
    strcpy(data, other.data);
    return *this;
}
```

```
void prt() const {
    std::cout << data << std::endl;
};

int main() {
    MyString str1("Hello");
    MyString str2 = str1; //copy
constructor
    MyString str3;
    str3 = str1; //copy assignment
operator
    str1.prt(); str2.prt(); str3.prt();

    //Output: Hello
    return 0;
}
```



Rule of 3 – Example – Not followed

```
1  #include <iostream>
2  #include <cstring>
3
4  class OtherString {
5  private:
6      char* data;
7      // Pointer to dynamically allocated memory
8
9  public:
10     // Constructor
11     OtherString(const char* str = "") {
12         data = new char[strlen(str) + 1]; // Allocate memory
13         strcpy(data, str);                // Copy the input string
14     }
15
16     // Destructor
17     ~OtherString() {
18         delete[] data; // Free the dynamically allocated memory
19     }
```



Rule of 3 – Example (not followed)

```
24     // Method to print the string
25     void print() const {
26         std::cout << data << std::endl;
27     }
28 };
29
30 int main() {
31     OtherString str1("Hello");
32     OtherString str2 = str1;
33     // Default copy constructor is called (shallow copy)
34     str1.~OtherString();
35     //Manually called str1's destructor
36
37     str2.print(); // Output: Unknown
38
39     return 0;
40 }
```



Rule of 3 - Problems

Some of the problems resulting for not following the rule of 3 are:

- Double Deletion – deleting an object that has already been deleted due to default copy constructor/copy assignment operator behavior.
- Shallow Copy – creating shallow copies of objects, leading to a modification in one of them, modifying all of them.
- Memory Leaks – Improper deletion and release of resources may cause memory leaks.



Rule of 3 - Summary

- The rule of 3 states that: If a class needs to define one of the following:
 - Destructor,
 - Copy Constructor
 - Copy Assignment Operator

then it needs to explicitly define all 3 in order to have exception safe coding.

- Without following the rule of 3, problems such as
 - Double deletion
 - Shallow copy
 - Memory leaks

may happen.